

Agentic AI: Intelligent Task Execution System

MAJOR PROJECT REPORT

**Submitted in partial fulfillment of the requirement for the Degree of
Bachelors of Engineering in Computer Science & Engineering**

Submitted To:



[PARUL UNIVERSITY, VADODARA, GUJARAT (INDIA)]

Submitted By:

Shubham Paithane [2303051051245]

Adarsh Singh Rajput [2303051051092]

Under The Guidance of:

Mr. Meet Kumar Patel

ASSISTANT PROFESSOR

CSE

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PARUL INSTITUTE OF TECHNOLOGY VADODARA, GUJARAT
SESSION: AY 2026-2027**

Parul University
Parul Institute of Technology



(Session: 2026 -2027)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that **SHUBHAM PAITHANE, ADARSH RAJPUT** students of CSE VII Semester of “**Parul Institute of Technology, Vadodara**” have completed their Major Project titled “**Agentic AI – Autonomous Task Execution Assistant (Aizen)**”, as per the syllabus and has submitted a satisfactory report on this project as a partial fulfillment towards the award of degree of Bachelor of Technology in Computer Science and Engineering under Parul University, Vadodara, Gujarat (India).

Mr. Meet Kumar Patel
ASSISTANT PROFESSOR
CSE
Project Guide

Ms. Arnika Patel
CSE DEPARTMENT
PIT, Vadodara
Project Coordinator

Dr. Sumitra Menaria
Head (CSE)
PIT, Vadodara
Head of Department

External Examiner

Dr. Ruchi Srivastava
PRINCIPAL
PIT, Vadodara
Principal

DECLARATION

We the undersigned solemnly declare that the project report “**Agentic AI – Autonomous Task Execution Assistant (Aizen)**” is based on our own work carried out during the course of our study under the supervision of **Mr. Meet Kumar Patel, Assistant Prof., CSE, PIT.**

We assert the statements made and conclusions drawn are the outcomes of our own work. We further certify that

- The work contained in the report is original and has been done by us under the general supervision of our supervisor.
- The work has not been submitted to any other Institution for any other degree / diploma / certificate in this university or any other University of India or abroad.
- We have followed the guidelines provided by the university in writing the report.
- This report is an extension of our minor project “Agentic AI – Autonomous Task Execution Assistant”, upgraded and substantially rebuilt for the major project stage.

Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them in the text of the report and giving their details in the references.

SHUBHAM PAITHANE [2303051051245]

ADARSH SINGH RAJPUT [2303051051092]

ACKNOWLEDGEMENT

Behind any major work undertaken by an individual there lies the contribution of the people who helped him to cross all the hurdles to achieve his goal.

It gives us immense pleasure to express our sense of sincere gratitude towards our respected guide **Mr. Meet Kumar Patel**, Assistant Professor, for his persistent, outstanding, invaluable co-operation and guidance. It is our achievement to be guided under him. He is a constant source of encouragement and momentum that any intricacy becomes simple. We gained a lot of invaluable guidance and prompt suggestions from him during the entire project work. We will be indebted to him forever and we take pride to work under him.

We also express our deep sense of regards and thanks to **Dr. Sumitra Menaria**, Head of the Computer Science & Engineering Department, and to **Ms. Arnika Patel**, Project Coordinator, CSE Department, for their precious advice, guidance and leadership. We are also grateful to **Dr. Ruchi Srivastava**, Principal, Parul Institute of Technology, for providing the environment and resources that made this project possible.

Last but not the least, our humble thanks to the Almighty God.

Place: Vadodara

Date:

ABSTRACT

When we start a new coding project, most of the early time does not go into the actual idea, it goes into setup: picking a stack, creating the folder structure, writing config files, and getting a basic version running before we can even start on the real feature. Tools like Copilot help with autocomplete inside a file, but someone still has to decide the project structure and put all the pieces together by hand. In our minor project we proposed an **agentic assistant** that could take one instruction in plain English and work out the steps needed to do it, instead of the user typing out each step separately. That project stayed mostly conceptual. For the major project we picked one specific area to actually build this idea in: generating and updating software projects, because it is easy to check whether the agent did its job properly, either the project it built runs, or it doesn't.

We call the system **Aizen**. It is a web-based coding assistant that runs in the browser. When the user types an instruction, the backend first figures out what kind of request it is using a simple intent classifier, whether it is just a question, a request to build something new, or a request to change a project that already exists in the workspace. Build and refine requests go to a project-builder function that asks a large language model (we use Groq) to return a project as a structured response, a list of files with their content and a short explanation, across ten stacks that we support, from a plain HTML page to a full React + Flask app. If that model call fails for any reason, a deterministic fallback kicks in and generates a basic working template instead, so the user never gets an empty response. After the first draft of a project is ready, an agent loop takes over: it can list files, read one, rewrite one, or run a validation command, and repeats this for a limited number of steps until it decides the project is in a good enough state, instead of just generating once and stopping.

Around this core loop we built an authenticated workspace with JWT-based login and per-user chat history, an in-browser Monaco code editor with a live preview for the generated files, a sandboxed terminal to run commands, a one-click GitHub push, a small template gallery, a usage dashboard, and a separate SQL Assistant panel that converts an English question into a SQL query. The backend is Python with Flask, the frontend is React with Vite, and SQLite stores user accounts and chat history.

This report covers why we upgraded from the minor project, a survey of similar agentic and code-generation tools and where Aizen fits compared to them, the methodology and module breakdown of the system, its system requirements, the interface as it is currently implemented, and the problems we ran into and fixed while building it, along with what we would still like to add.

INDEX

CHAPTER	TOPIC	PAGE NO.
Chapter 1	INTRODUCTION	11
1.1	Overview	11
1.2	Problem Statement	12
1.3	Objective of Project	13
1.4	Applications or Scope	14
1.5	Organization of Report	14
1.6	From Minor Project to Major Project	15
Chapter 2	LITERATURE SURVEY	17
Chapter 3	METHODOLOGY	21
3.1	Background / Overview of Methodology	21
3.2	Project Platforms used in Project	21
3.3	Proposed Methodology	22
3.4	Project Modules	24
3.5	Diagrams (Architecture, DFD, Use Case)	25
Chapter 4	SYSTEM REQUIREMENTS	29
4.1	Software Requirements	29
4.2	Hardware Requirements	29
Chapter 5	EXPECTED OUTCOMES...(with GUI)	31
Chapter 6	CONCLUSION & FUTURE SCOPE	36
6.1	Conclusion	36
6.2	Future Work	37
Chapter 7	REFERENCES	39

LIST OF TABLES

Table No.	Table Description	Page No.
Table 1.1	Minor project vs. major project	15
Table 4.1	Software requirements	29
Table 4.2	Client-side hardware requirements	29
Table 4.3	Server-side hardware requirements	30

LIST OF FIGURES

Figure No.	Figure Description	Page No.
Figure 3.1	System architecture of Aizen	25
Figure 3.2	DFD Level 0 - context diagram	26
Figure 3.3	DFD Level 1 - detailed process breakdown	27
Figure 3.4	Use case diagram - Aizen	28
Figure 5.1	Aizen landing page	31
Figure 5.2	Sign-up / login interface	32
Figure 5.3	Chat-based build workspace	33
Figure 5.4	Code editor and live preview	33
Figure 5.5	Analytics dashboard	34
Figure 5.6	SQL Assistant panel	35

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CLI	Command Line Interface
4	CORS	Cross-Origin Resource Sharing
5	CSS	Cascading Style Sheets
6	DFD	Data Flow Diagram
7	GUI	Graphical User Interface
8	HTML	HyperText Markup Language
9	HTTP/HTTPS	HyperText Transfer Protocol (Secure)
10	IDE	Integrated Development Environment
11	JSON	JavaScript Object Notation
12	JWT	JSON Web Token
13	LLM	Large Language Model
14	REST	Representational State Transfer
15	SSE	Server-Sent Events
16	SQL	Structured Query Language
17	UI	User Interface
18	URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 Overview

Aizen is a web-based agentic AI system that lets a user complete a software task by typing one natural-language instruction, instead of doing everything manually step by step. It is built around two ideas that we first proposed, in a more general and mostly conceptual form, in our minor project:

- **Autonomous Task Planning** – the agent figures out what the user is actually asking for, and for build or refine requests, works out what files the project needs before it writes anything.
- **Autonomous Task Execution** – the agent generates the files, writes them into a workspace, and can go back and check, re-run and fix its own output through a bounded loop instead of just generating once and stopping.

In the minor project “task” was kept pretty general and abstract. For the major project we narrowed it down to one domain, generating and maintaining software projects, because this gives us a simple way to check whether the agent actually did what it was supposed to: either the project it built runs, or it doesn't.

We use the word “agentic” here for a specific reason, not just as a buzzword. A normal AI assistant is reactive, it answers exactly what you ask and then stops, and the user still has to break the goal into steps, decide what each step should produce and put the pieces together. An agentic system is given a goal instead of one step, and it decides on its own what sequence of actions will get there, and it can look at the result of one action before choosing the next one. That is roughly what Aizen does: the intent classifier figures out what kind of request has come in, the project builder turns that into an initial set of files, and the tool-use loop lets the agent look at what it just produced, spot problems, and fix them, all without the user re-typing anything or walking it through each step. The user doesn't need to know any of this terminology though, from their side it just means they type one sentence and get back a working project they can open and edit, instead of a pile of code snippets they have to assemble themselves.

We should also be clear about what Aizen is not trying to be. It's not meant to replace a full IDE, and we are not trying to compete with big, general-purpose coding assistants on the number of languages they support or on enterprise features. We kept the scope narrow on purpose: take one instruction, produce or edit a small-to-medium multi-file project across a fixed set of common stacks, and keep the whole loop, from instruction to a runnable, previewable result, visible inside one browser tab. Keeping it narrow is what made it possible for two of us to actually build, test and understand the whole thing within our project timeline, while still covering every stage of a real agentic pipeline: classification, planning, generation, self-correction and delivery.

1.2 Problem Statement

A student or developer who wants to try out an idea quickly usually gets slowed down by the same repetitive setup work, no matter how simple the idea actually is: pick a stack, scaffold the project, wire up a couple of config files, and only then start on the part that actually matters.

- **For individual users** – existing AI assistants mostly answer one question at a time. Going from an idea to a runnable multi-file project still means copying code out of a chat window by hand, one file at a time.
- **For students and hobby developers** – the tools that do generate whole projects (IDE-integrated agents, hosted app builders) tend to lock you into a specific editor, a subscription, or a hosting platform, and they usually don't show the reasoning or intermediate steps.
- **For our own minor project** – the design was only at the diagram level. It proposed an agent loop but never actually tested it against a domain narrow enough to check. So we still didn't know whether the idea would hold up once we had to deal with real prompts, real failures and real output.

These three points are really the same gap seen from different angles. For an individual user it's friction, the intelligence to write any one file already exists in a chat window, but putting many of those answers together into one working project is left entirely to the human. For a student or hobbyist it's lock-in and opacity, the tools that remove that friction usually do it by tying you to one paid product and hiding how the generation actually works. And for our own minor project it was just unverified ambition, it's fairly easy to draw a diagram of an agent that plans and executes,

but actually building it and watching it fail in ways the diagram never predicted is a different thing entirely.

So there was a clear need for a system that takes one instruction, plans and generates a working project across more than one stack, lets the user see and edit what was produced, and can recover when a generation step fails, all from inside an ordinary browser tab.

1.3 Objective of Project

The objectives carried over and refined from the minor project are:

- To build an agent that reliably classifies a user's natural-language instruction into a conversational reply, a new build request, or a refinement of an existing project.
- To implement autonomous project generation across a meaningful range of technology stacks (React, Flask, Node/Express, plain HTML, Python, Java, C++ and PHP), rather than one hard-coded template.
- To give the agent a bounded tool-use loop, so it can inspect its own output (list files, read a file, rewrite a file, run a command) and improve a result instead of only producing it once.
- To keep the system usable when the underlying language model call fails, by falling back to a deterministic template so the user always receives something they can act on.
- To provide an integrated browser-based environment, code editor, live preview, execution sandbox and one-click GitHub deployment, so nothing has to be copied out of the chat window by hand.
- To show, with a working implementation instead of just diagrams, that the agentic approach we explored in the minor project actually holds up when tested.

Each of these objectives lines up with a specific part of the system described later in this report: classification is Stage 2 in Section 3.3, multi-stack generation with fallback is Stage 3 and the module list in Section 3.4, the bounded tool-use loop is Stage 4, and the integrated browser environment is the set of screens covered in Chapter 5. Keeping this connection between an objective and an actual feature was useful while building, it gave us a way to check “does this

feature actually serve one of our stated goals” instead of just adding things because they seemed useful.

1.4 Applications or Scope

Since Aizen's core loop, one instruction in and a working project out, isn't tied to any one specific stack, the same mechanism is useful in a few different settings:

- **Rapid Prototyping** – turns a one-line idea (“build me a Flask API with auth”) into a working starter project in the time it takes to describe it.
- **Education** – lets a student compare how the same request is answered across different stacks, and look at the generated project structure instead of just one isolated code snippet.
- **Hackathons and personal projects** – removes the setup overhead that otherwise eats into limited build time.
- **Developer tooling** – the intent-classification and tool-use loop are general enough that they could be reused as the base for other, more specialised agents later.
- **Quick proof-of-concept work** – a team trying to decide if an idea is worth building out fully can get something clickable and runnable in a few minutes instead of planning a whole sprint for it.

None of these use cases need the user to already know which stack is “right” for the job, or to have a development environment set up before they can see if an idea is worth chasing. We kept the scope of this project limited on purpose, it does not cover production deployment, team collaboration, or long-term project maintenance. Those are listed as future work in Chapter 6 rather than something we are claiming to have built.

1.5 Organization of Report

This report is organized into the following chapters:

- **Introduction** – background, problem statement and objectives.
- **Literature Survey** – related agentic frameworks and code-generation tools, and the gaps between them and this project.

- **Methodology** – platforms, proposed methodology, modules and system diagrams.
- **System Requirements** – hardware and software needed to run Aizen.
- **Expected Outcomes** – the interface and behaviour of the implemented system.
- **Conclusion & Future Scope** – summary of what was achieved and what is planned next.
- **References** – papers, documentation and tools referred to during the project.

1.6 From Minor Project to Major Project

Our minor project, submitted under the same title, proposed an agentic assistant that could plan and execute a task from a single instruction, but it stopped at the design stage, the agent loop, the intent handling and the diagrams were all conceptual, and there was no working system we could point to and say “this is what it actually does.” Turning that proposal into Aizen wasn't just a matter of coding up the original diagrams as they were drawn. A lot of the actual engineering decisions in Chapter 3, especially the bounded tool-use loop, the deterministic fallback, and the SQL Assistant module, only came up once we were sending real prompts to a real model and had to deal with real failures, none of which showed up while the design still only existed on paper. The table below sets out what specifically changed on the way to this major project.

Table 1.1: Minor project vs. major project

Aspect	Minor Project	Major Project (this report)
Scope	A general-purpose task-execution agent, discussed at the level of an idea.	Narrowed specifically to software project generation and refinement, where success is checkable: the project either runs or it does not.
Implementation	Diagram-level design only; no working application was built or demonstrated.	A complete working web application, Aizen, built and used with real prompts.
Technology stack	Proposed only; no backend, frontend or database was implemented.	Flask + Groq LLM backend, React + Vite frontend, JWT authentication, SQLite, Monaco editor.
Agent behaviour	Single-pass planning described conceptually, with no bound on how it would behave.	A bounded tool-use loop (list, read, write, run) that inspects and corrects its own output over several steps.
Failure handling	Not addressed.	An explicit deterministic fallback plus visible debug logging when the underlying model call fails, found

		necessary after a real failure during development.
Diagrams	Generic DFD and use-case diagrams not tied to a specific implementation.	All diagrams redrawn to reflect Aizen's actual modules and data flow.
Evaluation	No way to test whether the agent "worked."	Directly testable: a build request either produces a running project or a clear, logged error explaining why not.
Feature set	Limited to the single task-execution loop described in the proposal; no auxiliary domain-specific assistants.	Extended with a dedicated SQL Assistant that turns an English question into a runnable SQL query, added as a self-contained panel and backend route on top of the existing architecture.

CHAPTER 2

LITERATURE SURVEY

Purpose: to review existing agentic-AI research and code-generation tools relevant to building a system that plans and executes multi-file software projects from a single instruction, and to identify what is still missing.

1. Background: why agentic AI matters

A normal AI assistant is reactive, it answers one instruction at a time and then stops. Agentic AI is a move toward systems that can hold on to a goal, plan a sequence of actions to reach it, and act with much less step-by-step supervision from the user. This is what our minor project tried to explore in general terms, and what this major project actually applies to software generation.

This matters for a practical reason, not just a conceptual one. With a reactive assistant, the user has to figure out all the sub-steps of their goal themselves, ask for each one separately, and keep track of how the pieces fit together. That's fine for a single well-defined question, but it doesn't scale to something like "build me a working project," which breaks down into a lot of small decisions: which stack to use, how to lay out the files, writing each file so it's consistent with the others, and checking that the whole thing actually runs. An agentic system takes on that breakdown work itself instead of leaving it to the user. This is the main thing we looked at when comparing other tools in this survey, how much of that planning work they actually take off the user's hands.

2. Major categories of agentic-AI approaches

Looking at the space broadly, agents can be grouped into a few categories, and each one trades off flexibility against predictability differently:

- **Rule-based / script-driven agents** – older automation that relies on fixed decision trees and pre-written scripts to run through a sequence of steps. Predictable and cheap to run, but they can't handle an instruction that wasn't already anticipated when the rules were written.
- **LLM-powered agents** – a large language model reads the instruction, comes up with a plan, and calls tools as needed instead of following a fixed script. This is the category Aizen falls into,

our intent classifier and project builder both rely on a hosted LLM to make sense of free-form instructions that a fixed rule set could never fully cover.

- **ReAct (Reason + Act)** – mixes short reasoning steps with tool calls, so the agent's next move is based on what the last one returned (Yao et al., 2022). Our tool-use loop (Section 3.3, Stage 4) works the same way, at every step the model is shown what its last action returned before it picks the next one.
- **Tool-augmented LLMs** – frameworks like LangChain and AutoGPT let a model call external APIs, browse the web, or write files as part of doing a task. These frameworks support a wide range of possible tools; we instead hard-coded a small fixed set (list, read, write, run) specific to project generation, which we talk about more as a gap in Section 6.

3. Code-generation-specific tools

Since this project narrows the general agentic idea down to software engineering specifically, it makes more sense to compare it against tools built for that narrower job rather than just general-purpose agent frameworks:

- **GitHub Copilot** – strong at line-level and function-level autocomplete inside an editor, but it does not plan or create a multi-file project structure on its own; the developer still owns the scaffolding.
- **Cursor and similar IDE-integrated agents** – can apply multi-file edits, but require the specific editor to be installed and are usually tied to a paid subscription.
- **Hosted prompt-to-app builders (e.g. v0.dev, bolt.new)** – generate a working front-end quickly but are generally tied to one stack and to the vendor's own hosting, with limited visibility into the generation loop itself.
- **Devin and comparable “autonomous software engineer” demos** – show that a long-running autonomous coding agent is possible, but access is limited and the internal architecture isn't published in detail.

Looking at these together, there's a pretty clear pattern: the closer a tool gets to actually autonomous, multi-file software generation, the more likely it is to be closed-source, behind a subscription, or tied to one vendor's editor and hosting. There isn't much in the open, inspectable

middle, a system where a student can actually read and follow the planning and self-correction steps, and that's roughly where we're trying to place this project.

4. Frameworks and general-purpose building blocks

Apart from tools aimed specifically at coding, we also looked at a few general-purpose agent frameworks while designing our own tool-use loop, even though we didn't use any of them directly as a dependency:

- **LangChain** – a widely used framework for combining LLM calls with tool use and memory. Its “agent executor” pattern, looping a model over a fixed tool set until it says it's done, is basically the same shape as our own loop. We implemented ours directly instead of through the framework so it would be easier for us to trace what's actually happening at each step.
- **AutoGPT / BabyAGI** – early demos of a model looping over its own output to chase a goal with very little human input. These showed us that letting a loop run unbounded is unreliable in practice, which is basically why we capped our own loop at a fixed number of iterations instead of letting the model decide when to stop on its own.
- **OpenAI Assistants API** – gives built-in thread memory and tool-calling on top of a hosted model.
- **Semantic Kernel (Microsoft)** – an SDK for building planning and memory around an LLM inside enterprise applications.

5. Challenges identified in the literature

Going through these tools and frameworks, a few failure modes kept coming up, and any agentic system, ours included, has to actually design around them instead of assuming they won't happen:

- **Hallucination** – a model can confidently return a plan, a file, or a tool call that is just plain wrong.
- **Infinite or repetitive loops** – an agent loop with no limit can get stuck repeating the same action without making progress, which is why our loop has a hard cap on iterations.
- **Context window limits** – long conversations or large generated projects can end up bigger than what a single model call can actually see at once.

- **Silent failure** – a lot of these tools don't clearly report when the model call underneath has failed. We ran into this ourselves during development, it turned out to be a real problem and not just a theoretical one (see Chapter 6).
- **Evaluation difficulty** – judging open-ended agentic behaviour is still hard; “did it build something reasonable” is a lot harder to score than a plain right-or-wrong answer.

Out of these, silent failure is the one we actually ran into ourselves, and we go into it as a real example in Chapter 6 rather than just describing it as a risk we read about.

6. Identified gaps and how this project addresses them

Most of the tools we looked at are either general-purpose, and so not really built for the specific, checkable job of producing a runnable project, or they're code-generation tools that lock you into a specific editor, stack, or hosting setup. There's a gap for a browser-based assistant that isn't tied to one stack, that takes one instruction all the way from planning to a runnable, editable project, and that fails gracefully instead of silently when a model call doesn't work. That's roughly the gap we're trying to fill with Aizen: it supports ten stacks from one interface, keeps the tool-use loop and validation step visible to the user in the chat log, and has an explicit fallback for when generation fails.

We want to be careful not to overstate this. We are not claiming Aizen beats commercial tools like Copilot or Cursor on raw code quality, it doesn't, and that was never the goal. What we're showing is that this specific combination, stack-agnostic generation, a visible bounded self-correction loop, and a fallback that doesn't fail silently, can all be built together into one browser-based, fairly lightweight application. That's a narrower claim than “this is a better coding assistant,” and it's the one we can actually back up with what we built.

CHAPTER 3

METHODOLOGY

3.1 Background / Overview of Methodology

Aizen uses a single-entry, chat-first design. Every request from the user, whether it's a greeting, a build instruction, or a follow-up change, goes to one endpoint and is first passed through an intent classifier before anything else happens. The classifier picks one of three paths: a normal conversational reply, a new project build, or a refinement of a project that already exists in the user's workspace. Having one entry point like this made the system easier to reason about, and it's also what made it easier to track down the failure we describe in Chapter 6, since every request goes through the same small set of stages before it reaches the language model.

We could have gone with separate, explicit buttons for “build a project” versus “ask a question” instead, which would have removed the need for a classifier altogether. We didn't do that because it would have pushed the same problem we talk about in Chapter 2 back onto the user, they'd have to already know what category their request falls into before typing anything. Keeping one text box and figuring out the intent behind the scenes keeps the interaction more natural, but it does mean the classifier has to actually work reliably, which is why we treat it as its own separate step in Section 3.3 instead of folding it into project generation.

3.2 Project Platforms used in Project

- **Frontend:** React 19, Vite 6, Zustand for state, Tailwind CSS v4, Monaco Editor for in-browser code editing, react-markdown with syntax highlighting for chat output, recharts for the analytics view, lucide-react for icons.
- **Backend:** Python 3 with Flask and Flask-CORS, PyJWT for session tokens, Werkzeug's bcrypt-based hashing for passwords, SQLite for account and chat history storage.
- **AI integration:** Groq's hosted chat-completion API, used both for the conversational path and for the structured project-generation prompts.
- **Tools:** Visual Studio Code, Git and GitHub for version control, the browser's own developer tools for debugging the SSE stream.

We picked each of these for a specific reason rather than just going with whatever was popular. React with Vite made more sense than a heavier framework because this is one long-lived page, not a multi-route site that needs server-side rendering, so Vite's fast dev server and simple build setup fit better than the extra machinery something like Next.js would bring. Flask made more sense than Django for a similar reason: the backend is a handful of REST and streaming endpoints wrapped around one agent loop, not a full content-management app, so Flask's smaller footprint kept the codebase manageable for the two of us. We used SQLite instead of a client-server database because we're running a single local Flask process per user rather than a hosted multi-tenant service, this is also why we've listed it as a scaling limitation in Chapter 6. We picked Groq mainly because of its fast response times on chat-completion requests, which matters a lot for the tool-use loop in Stage 4, where several model calls happen back-to-back within one response the user is waiting on.

3.3 Proposed Methodology

The way the system works can be broken down into six stages, from the moment the user hits enter to the moment a runnable project shows up in their workspace:

1. User Instruction Intake

The user types a natural-language instruction into the chat box. The request goes to the backend along with a short slice of the recent conversation, so the agent has some memory of what was said before. There are no separate form fields here, the instruction is taken exactly as typed, which is what lets the same text box handle chat, build and refine requests without the user having to pick a mode first.

2. Intent Classification

A lightweight classifier, based on keywords and patterns rather than a model call, decides whether the instruction is a greeting or general question (chat), a request to build something new (create), or a request to change the current workspace (refine). We kept this step model-free on purpose, so it's fast and doesn't depend on the AI provider being reachable. That also means if Groq's API happens to be down for a moment, the system can still tell what kind of request it got, even if it can't act on it yet.

3. Response Generation (chat path) or Project Generation (build / refine path)

For a chat-classified request, the backend sends the instruction, a short system prompt and the recent conversation memory to the Groq API and streams the reply back to the browser. For a build request, a separate project-builder function asks the model for a structured response, a project name, a blueprint label, a short explanation and a list of files with their content, and the files get written into a per-session workspace on disk. If this model call fails for any reason (provider unreachable, a malformed response, or some unexpected exception), a deterministic template generator kicks in and produces a small but complete starter project instead, matched to the blueprint it detected, so the user never gets back an empty response. A refine request goes through the same path but also gets the current contents of the workspace as context, so the model edits what's already there instead of starting from scratch.

4. Agent Tool-Use Loop

Once there's an initial scaffold, the agent goes into a bounded loop (capped at six to eight iterations) where it can pick one of four actions at each step: list the files in the workspace, read a specific file, rewrite a file, or run a short validation command. At every step the model is asked whether it wants to take one more action or call the result final. This is what makes Aizen behave more like something that checks its own work rather than a one-shot generator, it can notice a file is incomplete and fix it before replying. We capped the number of iterations because of the infinite-loop problem we mention in Chapter 2, without a hard limit, a model that keeps deciding "one more action" could in theory never hand control back.

5. Validation

Before sending the result back, the backend tries to run one safe, allow-listed command from the generated project's own suggested run commands (for example, checking that a script actually parses) and records what happened as a validation note shown to the user next to the generated files. This is a light check, not a full test suite, its job is just to catch the most obvious kind of failure, a file that doesn't even parse, quickly and cheaply. It doesn't guarantee the generated project is fully correct.

6. Result Delivery

The final explanation, the generated or updated files, and a manifest describing the project (its blueprint, its files, and the validation outcome) get streamed back to the browser over Server-Sent Events and shown in the chat as an inline artifact, with the files also available in the code editor,

live preview and terminal panels. We stream the response instead of waiting for everything to finish because a build request can take a few seconds once the tool-use loop runs through several iterations, and seeing it stream in tells the user the system is actually working instead of just stuck.

3.4 Project Modules

- **Authentication Module** – sign-up and login, JWT issuance, bcrypt password hashing, per-user chat history.
- **Intent Classification Module** – routes each instruction to chat, create, or refine.
- **Project Builder Module** – model-backed generation across ten blueprints (React app, Flask API, static landing page, Python CLI, full-stack React+Flask, Node/Express API, Java CLI, C++ CLI, PHP site, and a minimal starter), with a deterministic template fallback for each blueprint.
- **Agent Tool-Execution Module** – the bounded list/read/write/run loop used both to polish a freshly generated project and to apply targeted edits to an existing one.
- **Workspace & File Management Module** – materialises generated files on disk per session and maintains the project manifest.
- **Live Preview & Code Editor Module** – a Monaco-based editor with save support and an iframe preview for HTML-renderable projects.
- **Execution / Terminal Module** – runs a chosen file's code in a sandboxed process and returns its output.
- **Search Module** – a client-side search across generated file names and contents.
- **GitHub Deployment Module** – pushes the current workspace to a GitHub repository.
- **Template Gallery Module** – a small set of curated starting points a user can launch directly.
- **Analytics Module** – tracks how many projects and files have been generated and in which languages, shown as simple charts.
- **SQL Assistant Module** – a dedicated panel that turns a plain-English question, together with an optional table schema and target dialect, into a runnable SQL query with a plain-English explanation. Instead of fine-tuning a separate model on a question-to-SQL dataset, we reused

the same Groq-hosted model and just narrowed it to this one job with a dedicated system prompt and a strict output format.

These twelve modules aren't twelve separately deployed services, Aizen is one Flask application and one React application, and the module list above is really about splitting up responsibility inside that codebase rather than splitting it across processes or servers. Grouping it this way still helped a lot while we were building, because we could work on and test one part, say authentication, without needing the project-builder or agent-loop parts to be working yet.

3.5 Diagrams

System Architecture

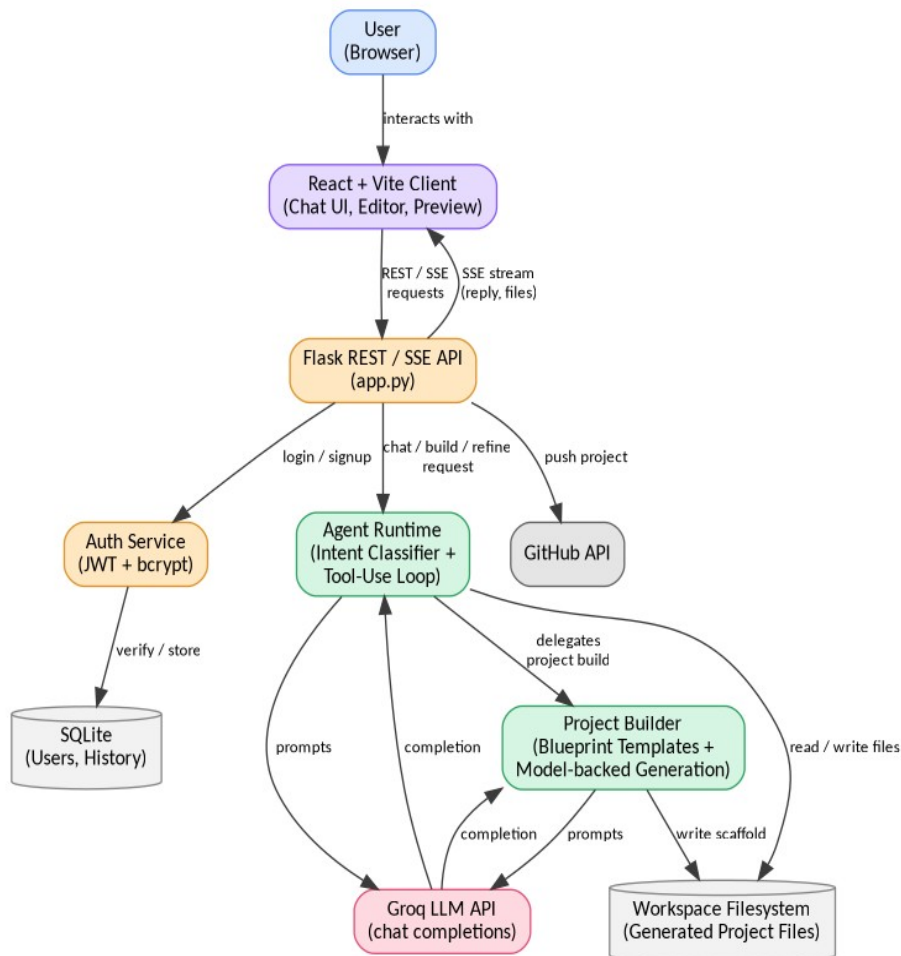


Figure 3.1: System architecture of Aizen

DFD Level 0 (Context Diagram)

Figure 3.2: DFD Level 0 – context diagram

DFD Level 1

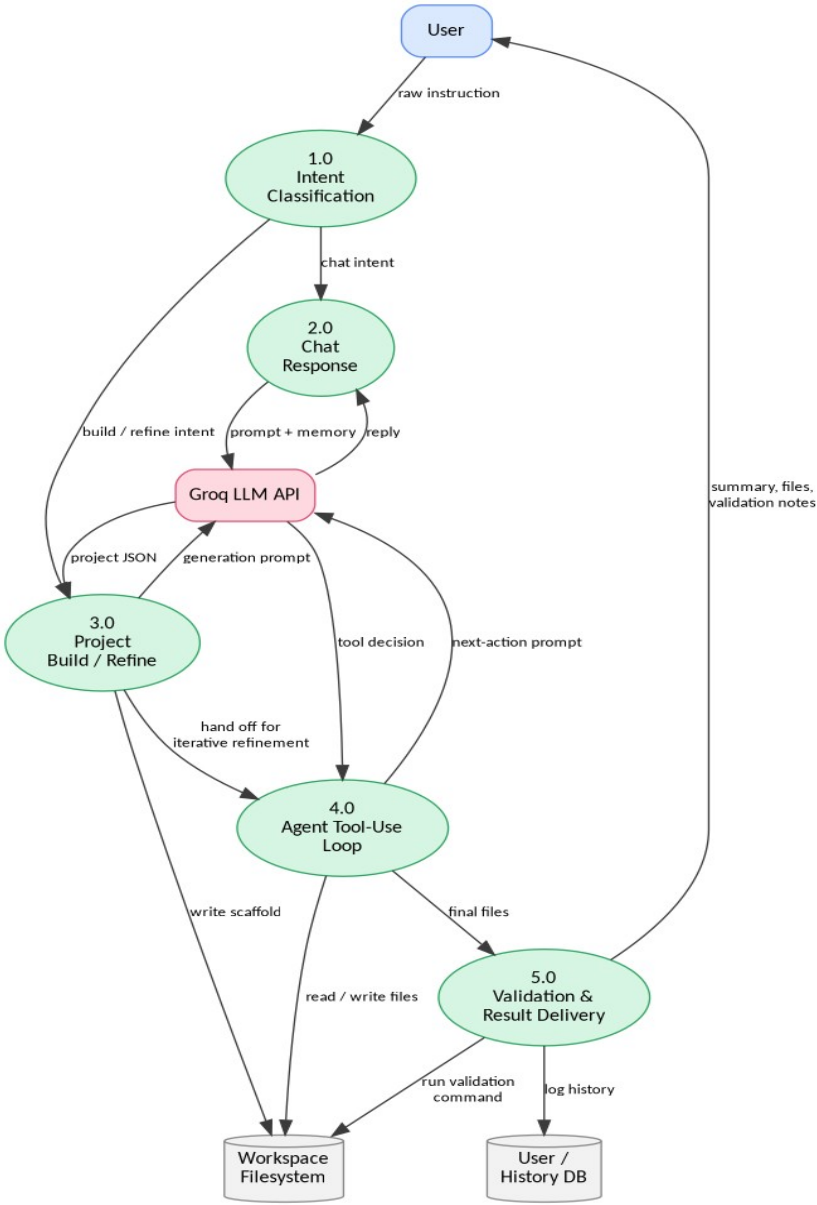


Figure 3.3: DFD Level 1 - detailed process breakdown

Use Case Diagram

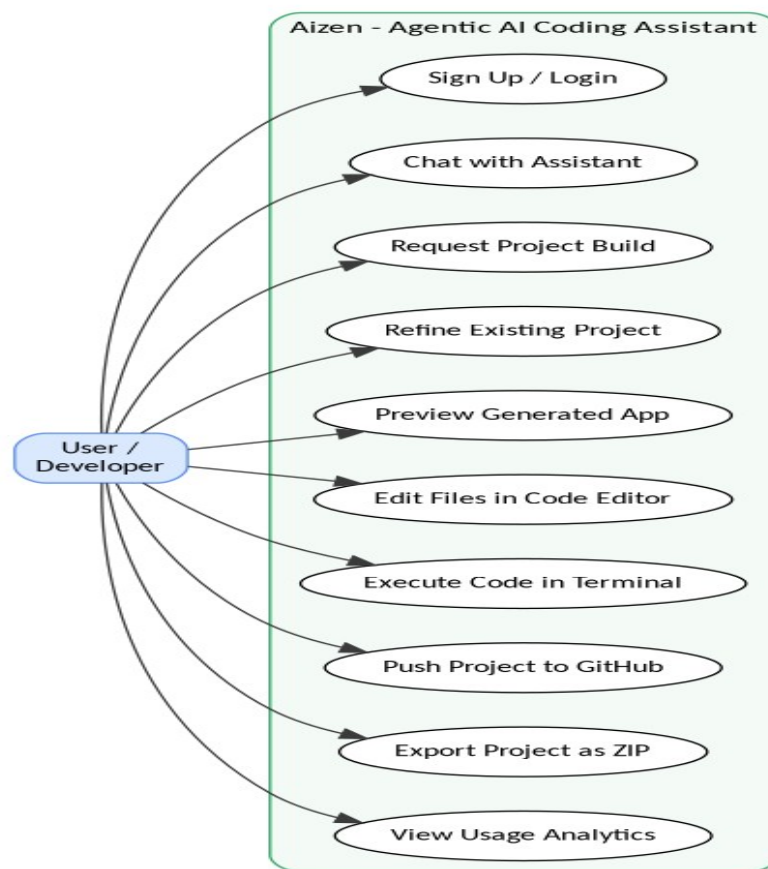


Figure 3.4: Use case diagram - Aizen

CHAPTER 4

SYSTEM REQUIREMENTS

The requirements below are split into software and hardware, and the hardware part is further split into what's needed on the client (the browser running the React app) and on the server (the machine running the Flask backend and the agent loop). We split it this way because the load on the two sides is actually quite different: the client mostly just renders the UI and runs the Monaco editor, while the server is where the model calls, file generation and validation actually happen.

4.1 Software Requirements

Table 4.1: Software requirements

Operating System	Windows / Linux / macOS
Frontend	React 19, Vite 6, Node.js 18+, Tailwind CSS v4
Backend	Python 3.10+, Flask 3.x, Flask-CORS
AI Integration	Groq chat-completions API
Database	SQLite
Authentication	PyJWT, Werkzeug (bcrypt hashing)
IDE / Tools	Visual Studio Code, Git / GitHub
Browser (Client)	Chrome / Edge / Firefox (latest versions)

4.2 Hardware Requirements

Neither the client nor the server needs particularly heavy hardware, since the actual model inference happens on Groq's hosted infrastructure and not on the user's machine or the local Flask server. What the local machine mainly needs is enough headroom to run a browser tab with an embedded code editor, and on the server side, enough space to hold several per-user workspaces on disk and make outbound API calls without slowing things down.

For Client (User side):

Table 4.2: Client-side hardware requirements

Processor	Intel i3 or above (or equivalent AMD)
RAM	Minimum 4 GB (8 GB recommended)
Storage	200 MB free space for cache / browser storage
Internet	Stable connection (2 Mbps or above)

For Backend Server (local Flask process):

Table 4.3: Server-side hardware requirements

Processor	Intel i5 or higher (or equivalent)
RAM	8 GB or above
Storage	1 GB minimum for workspaces and dependencies
Network	Outbound HTTPS access to the Groq API

CHAPTER 5

EXPECTED OUTCOMES...(WITH GUI)

This chapter goes through the interface screen by screen, roughly in the order a new user would actually see them: landing page, sign-up, the main chat/build workspace, the code editor and preview, and then the two extra tools, analytics and the SQL Assistant, that sit alongside the main workspace. The boxes below are placeholders marked for where a screenshot of the running application should go, since we did not have final screenshots ready at the time of writing this report.

5.1 Landing Page

The landing page introduces Aizen, says what it does in one line, and gives a direct path into the chat workspace. We kept it simple on purpose, it doesn't try to explain the agent loop or the module architecture from Chapter 3, it just gets a first-time visitor into the chat interface as fast as possible, since the product is easier to understand by using it than by reading about it.

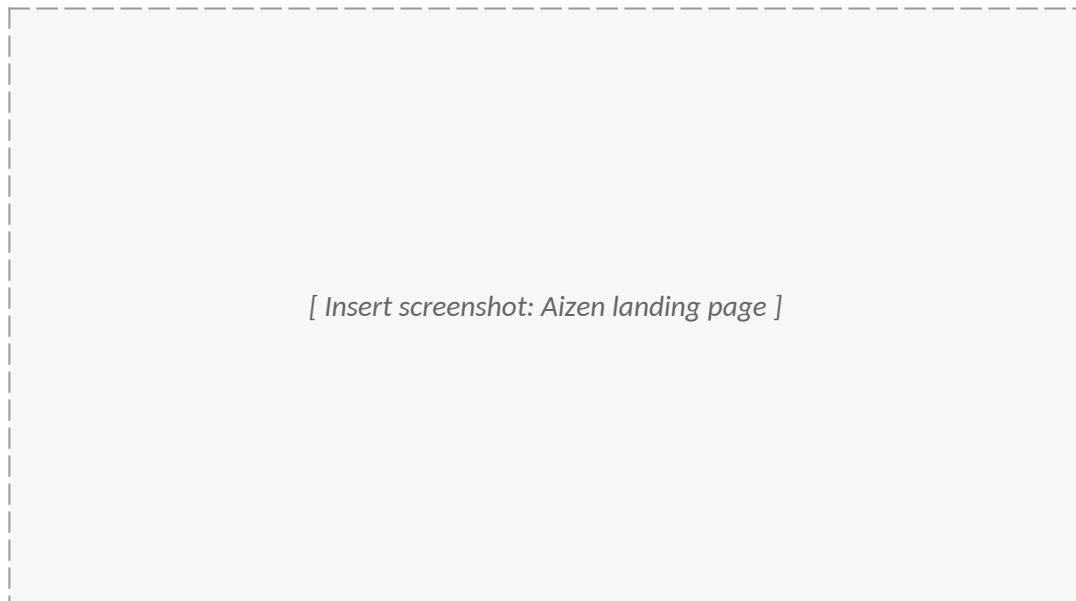


Figure 5.1: Aizen landing page

5.2 Sign-up / Login Interface

New users can create an account or log in with an existing one, and the session is then carried by a JWT stored on the client, which is what lets the sidebar show a user's own chat history when they

come back. Passwords are never stored as plain text, they're hashed with Werkzeug's bcrypt-based helper before being written to SQLite, so even someone with direct access to the database file wouldn't get a usable password out of it.

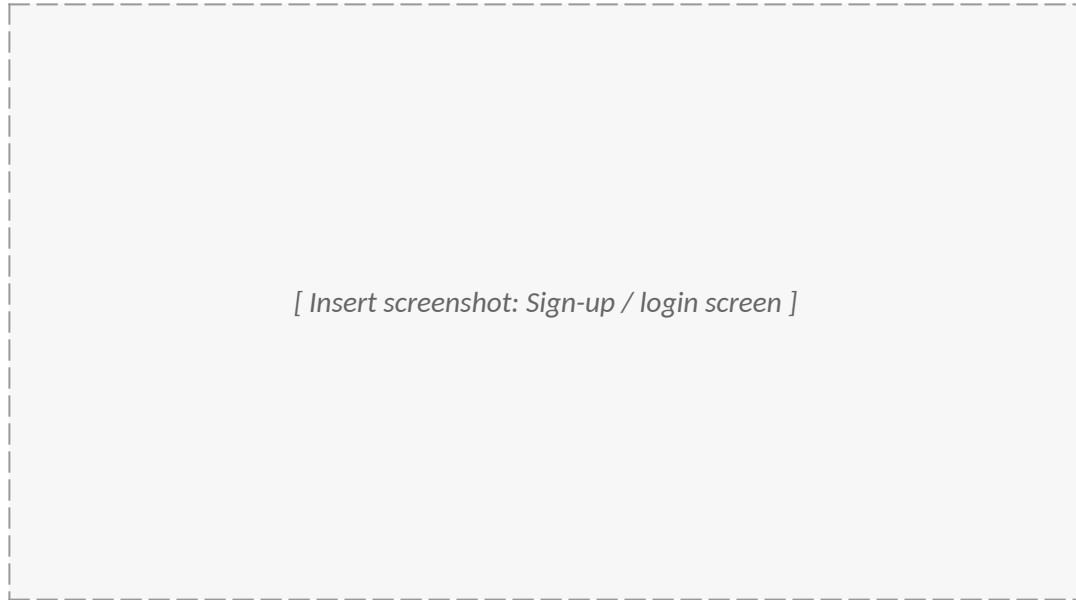


Figure 5.2: Sign-up / login interface

5.3 Chat-Based Build Workspace

The main workspace shows the conversation on the left, and once a project exists, the generated files, live preview, code editor, search and terminal panels on the right. The user can keep talking to the agent in the same thread to ask for changes, and the agent applies them to the same workspace instead of starting over from scratch. This is really the screen where the six-stage methodology from Chapter 3 becomes visible to the user, the streamed response, the inline project manifest and the validation note all show up here as the agent goes through classification, generation, the tool-use loop and delivery.

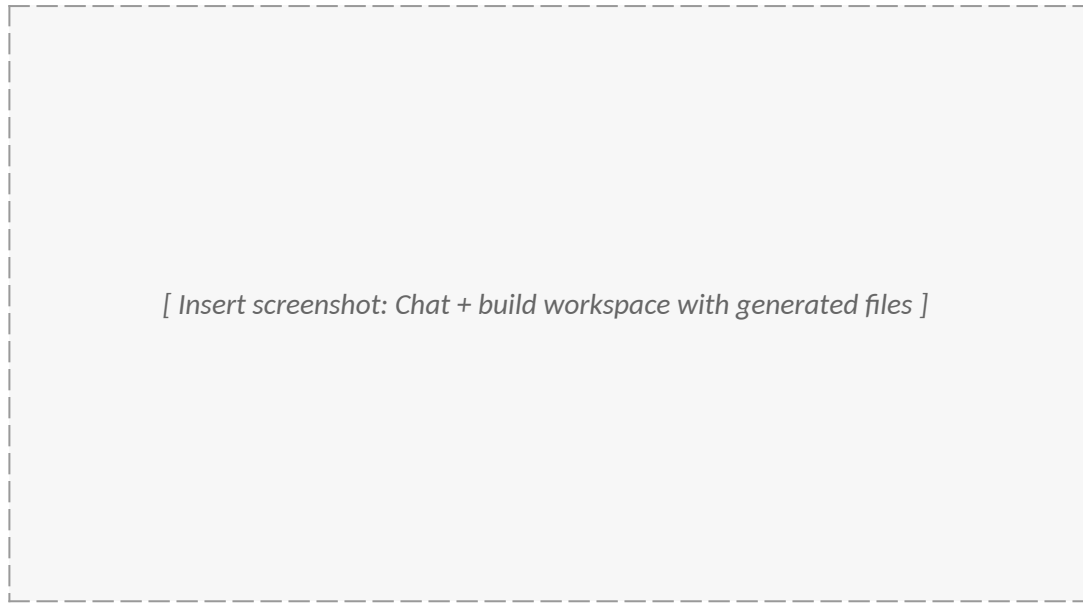


Figure 5.3: Chat-based build workspace

5.4 Live Code Editor and Preview

Generated files can be opened straight in an in-browser Monaco editor, edited, and saved back into the workspace; for HTML-renderable projects there's also an instant preview in a panel next to it. Using Monaco, the same editor that VS Code is built on, meant we got syntax highlighting, bracket matching and basic language support for free instead of having to build any of that ourselves.

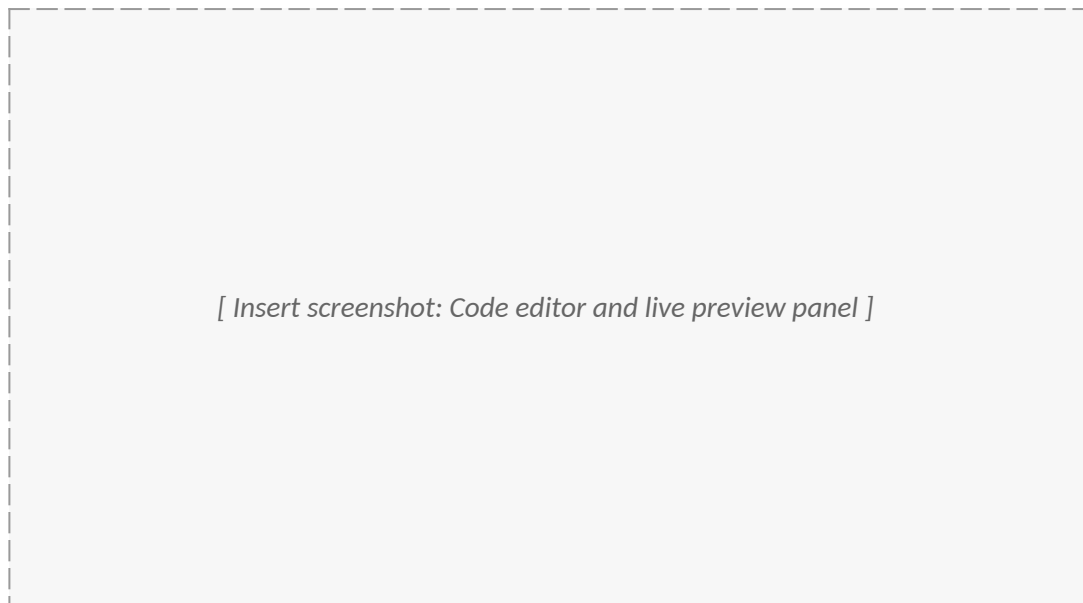


Figure 5.4: Code editor and live preview

5.5 Analytics Dashboard

A simple analytics view shows how many projects and files a user has generated and what mix of languages they used, mostly just as a way to see how the tool is actually being used over time. It's read-only, it just reports what happened, it doesn't try to recommend what to build next.



Figure 5.5: Analytics dashboard

5.6 SQL Assistant Panel

A separate panel lets the user optionally describe a table schema, pick a target SQL dialect, and ask a question in plain English, and the panel returns a syntax-highlighted SQL query along with a short explanation of what it does. It follows the same design as the other tool panels and opens from the same toolbar. The schema field is optional, so the assistant can still give a reasonable query just from the question, but giving it a schema noticeably improves the result since the model then has actual column and table names to work with instead of guessing them.

[Insert screenshot: SQL Assistant panel — English-to-SQL generation]

Figure 5.6: SQL Assistant panel

CHAPTER 6

CONCLUSION & FUTURE SCOPE

6.1 Conclusion

This project took the agentic-AI idea from our minor project, which was mostly just diagrams, and turned it into a system that actually works, in a domain narrow enough that we could tell whether it works or not: generating and maintaining software projects from a single instruction. What we actually completed is what's described through this report, the intent classifier, the project builder across ten stacks, the bounded tool-use loop, the validation step, the deterministic fallback, and the surrounding workspace with the editor, live preview, terminal, GitHub push and the SQL Assistant.

Building all of this end to end surfaced problems we never ran into while the idea was still on paper. The clearest example was a stretch during development where the assistant kept returning the same generic starter project no matter what we asked it to build. We eventually tracked this down to two things happening together: the intent classifier was missing some valid ways of phrasing a build request, and separately, a language-model call was failing silently and falling back to the template generator without logging why. Neither of these was obvious from the outside, we only found the real cause after we deliberately reproduced the failure and went through the backend logs, and then rewrote the exception handling so it actually reports what went wrong instead of just swallowing the error and returning a fallback. Fixing this taught us something the design phase didn't: from the user's side, an agent that fails silently looks exactly the same as one that never worked in the first place. So handling failure visibly ended up mattering just as much as the planning and generation logic.

As far as limitations go, Aizen is still built around a single local Flask process and SQLite, so it isn't set up for multiple users working on the same project at once, and the execution sandbox for running code is not isolated in a container yet. These are the two biggest gaps between what we have now and something closer to a deployable product, and they're the starting point for the future work listed below.

Overall, we think the project meets the objectives we set out in Chapter 1: instruction classification, multi-stack project generation with a working fallback, an iterative agent loop, and

an integrated browser environment for editing, running and shipping the result. It also answers the question our minor project left open, that this kind of agentic, tool-using approach can actually be built and can hold up under real use, as long as its failure paths are treated as seriously as the happy path.

6.2 Future Work

The items below are things we didn't get to, not because we forgot about them, but because we had to keep the project small enough for two people to finish in the time we had. Here's what we'd want to add next:

- **Smarter fallback and self-healing** – retry with a different model or a narrower prompt when the primary model call fails, instead of just dropping straight to the static template. Right now the fallback (Section 3.3, Stage 3) always gives the user something usable, but it doesn't try to recover the original request; retrying with a narrower prompt first would let more requests actually succeed on the AI-generated path.
- **Multi-agent collaboration** – split generation and validation between two agents, one that writes the code and one that specifically tries to break it, before showing the result to the user. This would turn the single validation command in Stage 5 into a more thorough, multi-step check.
- **Stronger execution sandboxing** – run generated and user-submitted code inside an isolated container instead of the host process, to cut down the risk from the terminal feature. This matters more as soon as the system is used by more than one trusted local user.
- **Voice-based interaction** – let the user speak instructions instead of typing them.
- **Team and workspace sharing** – let more than one user work on the same generated project at the same time, which would also mean moving off SQLite (already flagged as a limitation in Chapter 4) to something that handles concurrent access properly.
- **Mobile-friendly client** – a responsive or dedicated mobile version so the assistant isn't tied to a desktop browser.

Put together, these six points are a fairly natural next phase: make the single-user core we already have more solid, then grow outward toward collaboration and reach. That order, reliability first,

then scale, comes directly from the silent-failure problem above, it's more useful to make what already exists trustworthy than to keep adding new features on top of something we haven't fully verified yet.

CHAPTER 7

REFERENCES

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y., 2022. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629.
- [2] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. & Zhou, D., 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: Advances in Neural Information Processing Systems (NeurIPS 2022).
- [3] LangChain, 2024. LangChain Documentation. [online] Available at: <https://python.langchain.com> [Accessed 2026].
- [4] Significant Gravitas, 2023. AutoGPT: An Autonomous GPT-4 Experiment. [online] Available at: <https://github.com/Significant-Gravitas/AutoGPT> [Accessed 2026].
- [5] OpenAI, 2024. Assistants API Documentation. [online] Available at: <https://platform.openai.com/docs/assistants> [Accessed 2026].
- [6] Microsoft, 2024. Semantic Kernel Documentation. [online] Available at: <https://learn.microsoft.com/semantic-kernel> [Accessed 2026].
- [7] Groq Inc., 2026. Groq API Documentation. [online] Available at: <https://console.groq.com/docs> [Accessed 2026].
- [8] Pallets Projects, 2024. Flask Documentation. [online] Available at: <https://flask.palletsprojects.com> [Accessed 2026].
- [9] Meta Platforms Inc., 2025. React Documentation. [online] Available at: <https://react.dev> [Accessed 2026].
- [10] Vite, 2025. Vite Documentation. [online] Available at: <https://vitejs.dev> [Accessed 2026].
- [11] Microsoft, 2024. Monaco Editor Documentation. [online] Available at: <https://microsoft.github.io/monaco-editor> [Accessed 2026].
- [12] SQLite Consortium, 2024. SQLite Documentation. [online] Available at: <https://www.sqlite.org/docs.html> [Accessed 2026].

- [13] Jones, M., Bradley, J. & Sakimura, N., 2015. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force.
- [14] Pallets Projects, 2024. Werkzeug Documentation. [online] Available at: <https://werkzeug.palletsprojects.com> [Accessed 2026].
- [15] Tailwind Labs, 2025. Tailwind CSS Documentation. [online] Available at: <https://tailwindcss.com/docs> [Accessed 2026].